PROMPTS.md

How I used AI to build TrustVault, including the places where it was wrong and what I did about it.

I built this with an AI assistant (Claude) for code, explanations and debugging, working on a Windows laptop with VS Code. I ran everything myself, pasted real error screenshots back, and decided what to keep. Each entry below says what I asked, what came back, what was specifically wrong, and what I did instead.

Commit history: https://github.com/23jk1a4402-source/TrustVault/commits/main

Entry 1: 2026-10-24. I rejected a complete rewrite and kept my own code

What I asked: for the permissions, search and admin features, after I had already written login, upload, download and an audit page.

What it produced: a brand-new project: a different structure, separate template files, a longer `app.py` and a test suite, as one large download.

What was wrong: it replaced code I had written instead of extending it. I could not follow it, and I am expected to walk through my own code and change it live in an interview. A system I cannot explain is not a system I can defend.

What I did instead: I told it to drop the rewrite and work on my existing `app1.py` in small steps, one feature at a time, explaining each part. I only accepted a larger version after I had seen the earlier pieces working.

Entry 2: 2026-10-24. Styling instructions broke the application

What I asked: how to make the interface look professional.

What it produced: a stylesheet plus steps that assumed the page designs lived in a templates folder and a `static/style.css` file.

What was wrong: my application builds its pages from a string inside the Python file, so there was no such folder. Following the steps, the CSS ended up in the wrong place in `app1.py`. The page lost its styling and the terminal showed `sqlite3.OperationalError: near ">": syntax error`, with leftover `var(--bg)` lines that had no values. The assistant had not looked at my code before giving instructions.

What I did instead: I pasted my real file, and it replaced the whole file in one piece with the new styling instead of patching it. I then moved the styles into `static/style.css`.

Entry 3: 2026-10-24. Adding document sharing returned a 500 error

Area: access control.

What I asked: for per-document read and edit permissions, owner grant and revoke, and a permission check on download.

What it produced: a permissions table, a single `can_access()` function used by every document route, and share, revoke and edit pages.

What was wrong: after I added it, `/documents` returned Internal Server Error. The assistant's explanation was that my existing database already held an older permissions table, and `CREATE TABLE IF NOT EXISTS` does not change an existing table, so the new code met a table of a different shape. It had shipped a schema change with no migration step and no warning.

What I did instead: I stopped the server, dropped the old permissions table (it held no real data), restarted, and the page worked. I then tested the real behaviour by hand: a second account could not open the first account's file, read access let it download, and removing access stopped the download straight away.

Entry 4: 2026-10-24. A corrupted database query took the documents page down

Area: access control (the query that decides which documents a person sees).

What I asked: for the final version of the code with roles, search and the admin pages.

What it produced: a very long code block in the chat for me to copy.

What was wrong: after I pasted it, `/documents` failed with `sqlite3.OperationalError: no such column: u.username`. In the terminal the query lines were visibly mixed together: the "my documents" query carried part of the "shared with me" query and was missing its `JOIN users u` line. Copying a long block through the chat had damaged it. The assistant's output was correct when it ran its own tests, so the fault was in delivery, and it had not warned me that long pastes can break.

What I did instead: I replaced the entire file from the downloaded `app.py` rather than patching the lines by hand, ran the application again, and confirmed the documents page and its search worked.

Entry 5: 2026-10-24. Any logged-in user could read everyone's audit log

Area: access control and the audit trail.

What I asked: for an audit log of what happens to documents.

What it produced: an audit-log page and a button on the dashboard that every logged-in user could use. It listed every user's actions, including the names of other people's files.

What was wrong: the log worked, but it was visible to the wrong people. A user could see which files others had uploaded, shared or deleted. For a platform whose purpose is that the wrong person never learns about a confidential document, this was a leak, and the log itself could still be changed by anyone with access to the database file.

What I did instead: I added an administrator role, restricted the audit pages to it on the server (not only by hiding the button), and logged refused attempts to open them. I made the log append-only with database triggers and added a hash chain, with a button that verifies it. I covered all of it with tests (`test_only_admin_can_open_audit_log_and_user_list`, `test_audit_log_refuses_edits_and_deletes`, `test_tampering_with_the_audit_log_is_detected`).

Entry 6: 2026-10-25. The command that was meant to make an account an administrator did nothing

Area: access control (roles).

What I asked: how to make one account an administrator, since the website deliberately cannot do it.

What it produced: a one-line Python command that runs an UPDATE on the users table and prints `done`.

What was wrong: it printed `done` whether or not any row changed. When I listed the users, every account still showed `viewer` or `owner`, and the dashboard had no Audit Logs button. The command could not tell me it had matched nothing, so it reported success falsely. A second attempt by account number was clearer but still depended on me typing the right value.

What I did instead: I switched to a proper command built into the application, `python app.py make-admin <email>`. It checks that the account exists, says plainly if it does not, writes the role change to the audit log, and prints a confirmation. After running it the dashboard showed signed in as admin with the Audit Logs and Users buttons.

Instructions I reused across the week

These are the standing instructions I kept giving, because they changed how useful the answers were.

1. **"Keep my code and change it in small steps."** Never replace a whole file without showing the earlier pieces working first, and say where each change goes.
2. **"Explain each part simply, step by step."** I asked for the explanation in Telugu, then wrote the English documents from what I understood. The AI also helped me format the README and PDF and walked me through the Git and GitHub steps.

3. **"Tell me how to test it."** After each feature I asked for a quick manual check (a second browser window as another user) before moving on.
4. **"Look at my real screenshot or file."** I pasted terminal output and screenshots of errors instead of describing them. This corrected several of its wrong guesses (entries 2 and 4).

How I checked the AI's work

I did not accept a feature until I had seen it work: two accounts in two browser windows, a direct URL typed for a document that was not mine, access removed while the other window was open, and the audit page opened as an ordinary user and as an administrator. Those same checks are now automated in `tests/test_security.py`.